

LoveYou Use-Case Specification

Assignee	Reviewer	Editor
Hoàng Tấn	Minh Hoàng	Nghia

1. Authentication & Authorization (FG-01)

UC01: Sign Up

- **Use Case ID:** UC01
- **Use Case Name:** Sign Up
- **Actor(s):** Guest User (Primary)
- **Description:** Allows a guest user to create a new registered account on the LoveYou platform using an email address, password, full name, gender, and date of birth.
- **Preconditions:** The guest user is not currently logged into any active account.

Basic Flow (Main Success Scenario)

1. The guest user navigates to the Sign-Up page.
2. The system displays the account registration form requesting email, password, confirm password, display name, gender, and date of birth (must be 18+).
3. The guest user enters the required registration details and submits the form.
4. The system validates the input fields (email syntax, password complexity, age threshold ≥ 18).
5. The system checks whether the email is already registered in the system database.
6. The system hashes the user's password using `bcrypt` with cost factor ≥ 12 .
7. The system creates a new user account with state `PENDING_ONBOARDING` or `ACTIVE`.
8. The system generates a JWT authentication token session and logs the user in automatically.
9. The system redirects the user to the Onboarding wizard (`UC11`).

Alternative Flows

- **AF01.1: Invalid Input Data**
 - Step 4a: System detects invalid email syntax, weak password (under 8 characters, lacking required characters), or date of birth under 18 years old.
 - Step 4b: System displays specific inline error messages explaining the validation rules.
 - Step 4c: The user corrects the invalid inputs and resubmits.
- **AF01.2: Email Already Registered**
 - Step 5a: System finds an existing record with the provided email address.
 - Step 5b: System alerts the user: "This email is already associated with an account."
 - Step 5c: System provides options to log in or reset password (`UC04`).
- **AF01.3: Password Confirmation Mismatch**

- Step 4a: Password and confirm password fields do not match.
 - Step 4b: System displays an error message "Passwords do not match".
 - Step 4c: User re-enters password confirmation.
 - **Postconditions:** A new user record is persisted in the database with hashed credentials, and the user is logged in with an active session.
 - **Special Requirements:**
 - Password hashing must adhere to **SEC-01** (**bcrypt** cost factor 12).
 - Rate limiting: max 10 sign-up requests per minute per IP (**SEC-17**).
 - **UI Prototype Reference:** [!\[Sign Up Screen\]\(prototypes/uc01_sign_up.png\)](#)
-

UC02: Log In

- **Use Case ID:** UC02
- **Use Case Name:** Log In
- **Actor(s):** Guest User / User / Admin (Primary)
- **Description:** Authenticates a registered user or administrator using email and password, establishing an authenticated JWT session.
- **Preconditions:** The user account exists in the database.

Basic Flow (Main Success Scenario)

1. The user opens the Log In page.
2. The system renders the login form (email and password).
3. The user enters their credentials and clicks "Log In".
4. The system validates that input fields are non-empty.
5. The system fetches the account record and verifies the password hash using **bcrypt**.
6. The system triggers **UC05** (Manage Session) to issue a JWT access token (15-min TTL) and refresh token (7-day TTL).
7. The system triggers **UC06** (Authorize Access by Role) to evaluate account role (**USER** vs **ADMIN**).
8. If the user role is **USER**, system redirects to the Candidate Discovery feed or Onboarding if uncompleted. If **ADMIN**, system redirects to Admin Dashboard (**UC46**).

Alternative Flows

- **AF02.1: Incorrect Credentials**
 - Step 5a: Password hash verification fails or email does not exist.
 - Step 5b: System increments the failed login attempt counter for the email/IP.
 - Step 5c: System returns generic error: "Invalid email or password" (to prevent account enumeration).
- **AF02.2: Account Locked (SEC-03)**
 - Step 5a: System detects 5 consecutive failed login attempts within the window.

- Step 5b: System locks the account for 15 minutes and displays message: "Account temporarily locked due to multiple failed login attempts. Please try again in 15 minutes or reset your password."
 - **AF02.3: Account Blocked or Deactivated**
 - Step 5a: Account status is **BLOCKED** or **DEACTIVATED**.
 - Step 5b: System displays account status notification and denies session creation.
 - **Postconditions:** The user is authenticated and issued a secure JWT token pair; session metrics are updated.
 - **Special Requirements:**
 - JWT tokens must satisfy **SEC-02** (access token 15 min TTL, refresh token 7 days TTL).
 - Enforce lockout after 5 failures (**SEC-03**).
 - **UI Prototype Reference:** [!\[Log In Screen\]\(prototypes/uc02_log_in.png\)](#)
-

UC03: Log Out

- **Use Case ID:** UC03
- **Use Case Name:** Log Out
- **Actor(s):** User / Admin (Primary)
- **Description:** Terminate the active authenticated session, invalidating refresh tokens and clearing client session state.
- **Preconditions:** User is currently logged into the system with an active session.

Basic Flow (Main Success Scenario)

1. The user clicks the "Log Out" button from the main header or profile settings dropdown.
2. The user client sends a request to the backend log-out endpoint with the refresh token.
3. The backend invalidates/blacklists the refresh token in storage.
4. The client clears JWT tokens from local storage / HTTP-only cookie storage.
5. The system redirects the user to the Landing / Login page.

Alternative Flows

- **AF03.1: Network Connection Failure**
 - Step 2a: Client network request fails before reaching server.
 - Step 2b: Client clears local storage tokens locally, alerts user, and redirects to Login page.
 - **Postconditions:** The user session is terminated; protected endpoints are no longer accessible without re-authentication.
 - **Special Requirements:** Response time < 500ms (**PERF-03**).
 - **UI Prototype Reference:** [!\[Log Out Screen\]\(prototypes/uc02_log_in.png\)](#)
-

UC04: Reset Password

- **Use Case ID:** UC04
- **Use Case Name:** Reset Password
- **Actor(s):** User (Primary)
- **Description:** Allows a user who forgot their password to request a single-use reset token sent to their registered email and set a new password.
- **Preconditions:** User account exists.

Basic Flow (Main Success Scenario)

1. User selects "Forgot Password?" on the login page.
2. System displays prompt for registered email address.
3. User submits registered email address.
4. System validates email format and verifies user existence.
5. System generates a secure single-use reset token (30-minute expiration) and sends a reset link to the email.
6. User opens email and clicks the reset link.
7. System validates token authenticity and expiration.
8. System renders "New Password" form.
9. User inputs new password and password confirmation.
10. System verifies password complexity, hashes password via **bcrypt**, updates user record, and invalidates reset token.
11. System displays success message and redirects to Log In (**UC02**).

Alternative Flows

- **AF04.1: Non-Existent Email**
 - Step 4a: Email address is not in system.
 - Step 4b: System shows generic message: "If this email is registered, a password reset link has been sent" (prevents email enumeration).
- **AF04.2: Expired or Invalid Reset Link (SEC-04)**
 - Step 7a: Token is expired (> 30 mins) or already used.
 - Step 7b: System displays error: "Reset link has expired or is invalid" and prompts user to request a new link.
- **Postconditions:** User password is updated in database; reset token is marked spent.
- **Special Requirements:** Reset links expire within 30 mins (**SEC-04**).
- **UI Prototype Reference:** ![Reset Password Screen](prototypes/uc02_log_in.png)

UC05: Manage Session

- **Use Case ID:** UC05
- **Use Case Name:** Manage Session

- **Actor(s):** System (Primary), User / Admin (Secondary)
- **Description:** Handles automatic JWT token validation, refresh, and session state maintenance across API requests. Included by UC02.
- **Preconditions:** User initiates authentication or authorized API call.

Basic Flow (Main Success Scenario)

1. System receives API request containing JWT Access Token header.
2. System validates JWT signature, issuer, and expiration time.
3. If access token is valid, system permits API route execution.
4. When access token expires, client issues token refresh request using valid Refresh Token.
5. System verifies refresh token, issues new Access Token, and updates session metrics.

Alternative Flows

- **AF05.1: Expired Refresh Token**
 - Step 5a: Refresh token has exceeded 7 days TTL.
 - Step 5b: System rejects refresh request with 401 Unauthorized, forces log out (UC03).
 - **AF05.2: Invalid Token Signature**
 - Step 2a: Token signature verification fails (tampering).
 - Step 2b: System logs security warning and returns HTTP 401.
 - **Postconditions:** Active session maintained securely or cleanly terminated on invalid token.
 - **Special Requirements:** JWT verification < 50ms overhead.
 - **UI Prototype Reference:** ![Session State Screen](prototypes/uc02_log_in.png)
-

UC06: Authorize Access by Role

- **Use Case ID:** UC06
- **Use Case Name:** Authorize Access by Role
- **Actor(s):** System (Primary), User / Admin (Secondary)
- **Description:** Enforces Role-Based Access Control (RBAC) preventing unauthorized access to role-restricted endpoints (e.g. Admin endpoints). Included by UC02.
- **Preconditions:** Request has passed session token validation (UC05).

Basic Flow (Main Success Scenario)

1. System reads target route permissions (e.g. `/api/v1/admin/*` requires `ROLE_ADMIN`).
2. System extracts user role claim from validated JWT payload.
3. System verifies that user role meets or exceeds required route role.
4. System routes request to appropriate backend handler.

Alternative Flows

- **AF06.1: Insufficient Permissions (SEC-11)**
 - Step 3a: Non-admin user attempts access to admin endpoint.
 - Step 3b: System rejects request with HTTP 403 Forbidden.
 - **Postconditions:** Protected resources accessed only by authorized roles.
 - **Special Requirements:** RBAC enforcement on 100% of endpoints (SEC-11).
 - **UI Prototype Reference:** ![Access Control Screen](prototypes/uc02_log_in.png)
-

2. User Profile Management (FG-02)

UC07: Edit Personal Information

- **Use Case ID:** UC07
- **Use Case Name:** Edit Personal Information
- **Actor(s):** Registered User (Primary)
- **Description:** Allows user to update profile details including display name, bio, home city, job title, and dating goals.
- **Preconditions:** User is logged in with an active session.

Basic Flow (Main Success Scenario)

1. User navigates to "Edit Profile" screen.
2. System loads and displays existing user profile data.
3. User modifies display name, biography text (max 500 chars), home city, job title, or height.
4. User clicks "Save Changes".
5. System validates field formats and lengths.
6. System updates profile record in database.
7. System displays success toast: "Profile updated successfully".

Alternative Flows

- **AF07.1: Exceeded Bio Length**
 - Step 5a: Bio text exceeds 500 characters.
 - Step 5b: System displays error: "Bio cannot exceed 500 characters".
 - **AF07.2: Database Persistence Error**
 - Step 6a: System experiences database connection timeout.
 - Step 6b: System displays error and prompts user to retry.
 - **Postconditions:** Updated profile information saved and visible to potential matches.
 - **Special Requirements:** Response time < 1s (PERF-04).
 - **UI Prototype Reference:** ![Edit Personal Info Screen](prototypes/uc01_sign_up.png)
-

UC08: Upload Photos

- **Use Case ID:** UC08
- **Use Case Name:** Upload Photos
- **Actor(s):** Registered User (Primary)
- **Description:** Allows user to upload, reorder, or delete up to 5 profile photos (max 5MB each, JPEG/PNG format).
- **Preconditions:** User is logged in.

Basic Flow (Main Success Scenario)

1. User opens photo gallery management tab under Edit Profile.
2. User selects local photo file or drag-and-drops photo file.
3. System checks MIME type (JPEG/PNG) and file size ($\leq 5\text{MB}$).
4. System uploads photo to cloud storage and saves signed URL asset reference.
5. System renders updated photo grid showing thumbnail preview.
6. User can drag to reorder avatar primary photo positioning.

Alternative Flows

- **AF08.1: File Too Large (PERF-12 / UX-03)**
 - Step 3a: File size exceeds 5MB.
 - Step 3b: System displays Vietnamese message: "Ảnh vượt quá 5 MB. Vui lòng chọn ảnh nhỏ hơn."
 - **AF08.2: Invalid File Type (SEC-16)**
 - Step 3a: File is executable or unsupported format (e.g. .exe, .gif).
 - Step 3b: System rejects upload and displays file type error.
 - **AF08.3: Maximum Photo Limit Reached**
 - Step 2a: Profile already has 5 photos uploaded.
 - Step 2b: System disables upload button and displays notice: "Maximum 5 photos allowed".
 - **Postconditions:** New photo stored and associated with user's profile card gallery.
 - **Special Requirements:** Max 5MB per image, up to 5 images (PERF-12).
 - **UI Prototype Reference:** ![Upload Photos Screen](prototypes/uc01_sign_up.png)
-

UC09: Manage Interest Tags

- **Use Case ID:** UC09
- **Use Case Name:** Manage Interest Tags
- **Actor(s):** Registered User (Primary)
- **Description:** Allows user to select and update interest tags (e.g. Travel, Coffee, Music, Board Games) from global catalogue.
- **Preconditions:** User is logged in; interest tag catalogue exists.

Basic Flow (Main Success Scenario)

1. User selects "Interests" section on profile edit view.
2. System displays categorized list of interest tags from catalogue.
3. User selects or deselects tags (minimum 3, maximum 10 tags).
4. User clicks "Save Interests".
5. System updates user's tag associations in database.
6. System confirms update with visual confirmation.

Alternative Flows

- **AF09.1: Fewer than 3 Tags Selected**
 - Step 4a: User selects fewer than 3 tags.
 - Step 4b: System prompts: "Please select at least 3 interest tags for better AI matching accuracy."
 - **Postconditions:** User interest tag preferences stored for AI compatibility calculations.
 - **Special Requirements:** UI response within 200ms.
 - **UI Prototype Reference:** ![Manage Interest Tags Screen](prototypes/uc01_sign_up.png)
-

UC10: Change Password

- **Use Case ID:** UC10
- **Use Case Name:** Change Password
- **Actor(s):** Registered User (Primary)
- **Description:** Allows an authenticated user to change password by entering current password and confirming new password.
- **Preconditions:** User is logged in.

Basic Flow (Main Success Scenario)

1. User enters Account Security settings and clicks "Change Password".
2. System renders fields: Current Password, New Password, Confirm New Password.
3. User inputs current password and new password pair.
4. System verifies current password against database **bcrypt** hash.
5. System validates new password meets strength requirements (≥ 8 chars).
6. System updates password hash and revokes old tokens.
7. System displays success notice: "Password updated successfully".

Alternative Flows

- **AF10.1: Incorrect Current Password**
 - Step 4a: Current password check fails.
 - Step 4b: System displays error: "Current password is incorrect".
- **Postconditions:** User password hash updated in database; security log updated.

- **Special Requirements:** Hash cost factor 12 (SEC-01).
 - **UI Prototype Reference:** ![Change Password Screen](prototypes/uc010_change_password.png)
-

3. Onboarding and Preference Setup (FG-10)

UC11: Complete Onboarding

- **Use Case ID:** UC11
- **Use Case Name:** Complete Onboarding
- **Actor(s):** Registered User (Primary)
- **Description:** Guided multi-step onboarding wizard for new users to configure profile essentials right after sign up. Includes UC12–UC16.
- **Preconditions:** User newly signed up (UC01), profile state PENDING_ONBOARDING.

Basic Flow (Main Success Scenario)

1. System automatically routes newly registered user to Onboarding wizard.
2. System executes UC12 (Upload Initial Photo).
3. System executes UC13 (Set Gender Preference).
4. System executes UC14 (Set Preferred Age Range).
5. System executes UC15 (Set Home City).
6. System executes UC16 (Select Initial Interest Tags).
7. User clicks "Finish Setup".
8. System updates user profile status to ACTIVE and routes to Candidate Discovery feed (UC18).

Alternative Flows

- **AF11.1: Skip Onboarding (UC17)**
 - Step 1a: User clicks "Skip for now" on onboarding step.
 - Step 1b: System triggers UC17 (Skip Onboarding), setting default preferences and warning user of lower AI score precision.
 - **Postconditions:** User profile is fully initialized and active for candidate discovery.
 - **Special Requirements:** Complete wizard within 3 minutes (UX-01).
 - **UI Prototype Reference:** ![Complete Onboarding Screen](prototypes/uc01_sign_up.png)
-

UC12: Upload Initial Photo

- **Use Case ID:** UC12
- **Use Case Name:** Upload Initial Photo
- **Actor(s):** Registered User (Primary)
- **Description:** Onboarding step for uploading mandatory primary avatar photo. Included by UC11.

- **Preconditions:** User in onboarding wizard flow.

Basic Flow (Main Success Scenario)

1. Onboarding wizard presents Photo Upload step.
2. User uploads primary profile picture.
3. System validates format and size, stores picture asset, and displays preview.
4. User proceeds to next onboarding step.

Alternative Flows

- **AF12.1: Photo Upload Error**
 - Step 3a: Upload fails due to network or validation.
 - Step 3b: System displays error message and prompts user to retry.
 - **Postconditions:** Primary avatar photo saved to user profile.
 - **Special Requirements:** File size $\leq$ 5MB (PERF-12).
 - **UI Prototype Reference:** ![Upload Initial Photo Screen](prototypes/uc01_sign_up.png)
-

UC13: Set Gender Preference

- **Use Case ID:** UC13
- **Use Case Name:** Set Gender Preference
- **Actor(s):** Registered User (Primary)
- **Description:** Onboarding step to select target match gender (Male, Female, Everyone). Included by UC11.
- **Preconditions:** User in onboarding flow.

Basic Flow (Main Success Scenario)

1. Onboarding presents Gender Preference step.
2. User selects preferred target gender.
3. System saves selection in temporary onboarding draft.

Alternative Flows

- **AF13.1: No Choice Selected**
 - Step 2a: User clicks Next without selecting.
 - Step 2b: System defaults preference to "Everyone".
 - **Postconditions:** Match gender filter saved.
 - **UI Prototype Reference:** ![Set Gender Preference Screen](prototypes/uc01_sign_up.png)
-

UC14: Set Preferred Age Range

- **Use Case ID:** UC14
- **Use Case Name:** Set Preferred Age Range
- **Actor(s):** Registered User (Primary)
- **Description:** Onboarding step to define minimum and maximum target age preferences using dual-slider. Included by **UC11**.
- **Preconditions:** User in onboarding flow.

Basic Flow (Main Success Scenario)

1. Onboarding presents Age Range selection step (e.g. range 18–99).
2. User adjusts range sliders (e.g. 20 to 28).
3. System validates $\text{min} \leq \text{max}$ and $\text{age} \geq 18$.
4. System records age filter preference.

Alternative Flows

- **AF14.1: Invalid Range**
 - Step 3a: Slider $\text{min} > \text{max}$.
 - Step 3b: System auto-adjusts min value to equal max value.
 - **Postconditions:** Target age boundary saved.
 - **UI Prototype Reference:** [!\[Set Preferred Age Range Screen\]\(prototypes/uc01_sign_up.png\)](#)
-

UC15: Set Home City

- **Use Case ID:** UC15
- **Use Case Name:** Set Home City
- **Actor(s):** Registered User (Primary)
- **Description:** Onboarding step to select home city/province from dropdown catalogue (e.g. Ho Chi Minh City, Hanoi, Da Nang). Included by **UC11**.
- **Preconditions:** User in onboarding flow.

Basic Flow (Main Success Scenario)

1. Onboarding presents City Selection step.
2. User searches or chooses city from dropdown.
3. System saves location setting.

Alternative Flows

- **AF15.1: Unlisted Location**
 - Step 2a: User location not found in dropdown.
 - Step 2b: System provides "Other / Nationwide" fallback option.
- **Postconditions:** Province/City level location recorded (**PRIV-03**).

- **Special Requirements:** Store city level only, no raw GPS (PRIV-03).
 - **UI Prototype Reference:** ![Set Home City Screen](prototypes/uc01_sign_up.png)
-

UC16: Select Initial Interest Tags

- **Use Case ID:** UC16
- **Use Case Name:** Select Initial Interest Tags
- **Actor(s):** Registered User (Primary)
- **Description:** Onboarding step to choose initial set of interest tags. Included by UC11.
- **Preconditions:** User in onboarding flow.

Basic Flow (Main Success Scenario)

1. Onboarding displays tag selector UI.
2. User taps 3 or more interest tags.
3. System highlights selected tags and enables "Complete" button.
4. User submits selection.

Alternative Flows

- **AF16.1: Insufficient Tags Selected**
 - Step 3a: Fewer than 3 tags selected.
 - Step 3b: System keeps "Complete" button disabled with guidance text.
 - **Postconditions:** Initial interest tags saved to user profile.
 - **UI Prototype Reference:** ![Select Initial Interest Tags Screen](prototypes/uc01_sign_up.png)
-

UC17: Skip Onboarding

- **Use Case ID:** UC17
- **Use Case Name:** Skip Onboarding
- **Actor(s):** Registered User (Primary)
- **Description:** Allows user to bypass remaining optional onboarding steps and complete setup with system defaults. Extends UC11.
- **Preconditions:** User is currently in the onboarding wizard (UC11).

Basic Flow (Main Success Scenario)

1. User clicks "Skip for now" link at top of onboarding view.
2. System displays prompt: "Skipping onboarding may decrease AI match score accuracy. Continue?"
3. User confirms "Yes, Skip".
4. System sets default preferences (Age: 18-99, City: User default, Gender: Everyone).
5. System sets account state to ACTIVE and routes user to discovery feed (UC18).

Alternative Flows

- **AF17.1: User Cancels Skip**
 - Step 3a: User selects "Go Back".
 - Step 3b: System resumes current onboarding step.
 - **Postconditions:** Account activated with default preference parameters.
 - **UI Prototype Reference:** ![Skip Onboarding Screen](prototypes/uc01_sign_up.png)
-

4. AI-Powered Smart Matching (FG-03)

UC18: View AI Match Suggestions

- **Use Case ID:** UC18
- **Use Case Name:** View AI Match Suggestions
- **Actor(s):** Registered User (Primary), AI Matching Service (Supporting)
- **Description:** Fetches and presents a candidate feed of recommended user cards sorted by AI compatibility score, accompanied by Vietnamese natural language explanations. Includes [UC19](#) and [UC20](#).
- **Preconditions:** User is logged in with an active profile.

Basic Flow (Main Success Scenario)

1. User opens the "Discover / AI Matches" main tab.
2. System retrieves candidate profiles matching user's basic gender, age, and location filters.
3. System invokes [UC19](#) (Calculate Compatibility Score) for top candidates.
4. System invokes [UC20](#) (Display Match Reasons) to attach Vietnamese match explanations.
5. System renders candidate cards displaying photo, display name, age, city, compatibility score badge (0-100), and AI match reasoning text.
6. User browses cards or swipes to interact ([UC22/UC23](#)).

Alternative Flows

- **AF18.1: No Candidate Profiles Found**
 - Step 2a: Search query returns zero candidate profiles matching filter criteria.
 - Step 2b: System displays empty state message: "No candidates found matching your criteria. Try broadening your age range or location filters."
 - Step 2c: System provides quick link to edit preferences.
- **AF18.2: AI Match Service Timeout / Fallback**
 - Step 3a: AI module call times out (> 5s).
 - Step 3b: System falls back to rule-based score calculation based on shared tag counts and renders profiles without delay.
- **Postconditions:** User views personalized candidate profiles sorted by compatibility.

- **Special Requirements:** AI score generation for 50 candidates < 5s (PERF-06).
 - **UI Prototype Reference:** ![View AI Match Suggestions Screen] (prototypes/uc18_ai_match_suggestions.png)
-

UC19: Calculate Compatibility Score

- **Use Case ID:** UC19
- **Use Case Name:** Calculate Compatibility Score
- **Actor(s):** System / AI Matching Service (Primary)
- **Description:** Evaluates multi-dimensional compatibility (shared tags, bio vector similarity, age, location) to produce a composite Compatibility Score from 0 to 100. Included by UC18.
- **Preconditions:** Active user profile and candidate profile data provided.

Basic Flow (Main Success Scenario)

1. System extracts feature vectors for user A and candidate B (interest tags vector, bio embeddings, location distance, age alignment).
2. System computes weighted similarity matrix:

$$\text{Score} = w_1 S_{\text{tags}} + w_2 S_{\text{bio}} + w_3 S_{\text{location}} + w_4 S_{\text{age}}$$

3. System normalizes score into an integer range $[0, 100]$.
4. System returns score value to calling component.

Alternative Flows

- **AF19.1: Missing Bio or Tag Data**
 - Step 1a: Candidate has empty bio or no tags.
 - Step 1b: System calculates score based on available demographic parameters with adjusted weights.
 - **Postconditions:** Integer score $[0, 100]$ generated for candidate pair.
 - **Special Requirements:** Standalone service architecture (MAINT-04).
 - **UI Prototype Reference:** ![Calculate Score Screen] (prototypes/uc18_ai_match_suggestions.png)
-

UC20: Display Match Reasons

- **Use Case ID:** UC20
- **Use Case Name:** Display Match Reasons
- **Actor(s):** User (Primary), System (Supporting)
- **Description:** Generates and displays localized Vietnamese explanation detailing why two users are compatible (e.g. "Cả hai cùng thích Du lịch và Cà phê tại TP.HCM"). Included by UC18.
- **Preconditions:** Compatibility score calculated (UC19).

Basic Flow (Main Success Scenario)

1. System receives shared interest tags and location overlap data from matching engine.
2. System formats natural language Vietnamese summary string (e.g. "Cùng yêu thích Du lịch, Cà phê và sống tại TP. Hồ Chí Minh").
3. System attaches explanation string to profile card view model.
4. System renders summary inside profile details view.

Alternative Flows

- **AF20.1: Minimal Overlap**
 - Step 1a: No shared tags found.
 - Step 1b: System displays default match reason: "Cùng độ tuổi phù hợp và có phong cách sống tương đồng."
 - **Postconditions:** Vietnamese match reason displayed on candidate card.
 - **UI Prototype Reference:** [!\[Display Match Reasons Screen\]\(prototypes/uc18_ai_match_suggestions.png\)](#)
-

UC21: Refresh Suggestions

- **Use Case ID:** UC21
- **Use Case Name:** Refresh Suggestions
- **Actor(s):** Registered User (Primary)
- **Description:** Allows user to manually trigger a pull-to-refresh action to get a fresh batch of candidate match suggestions. Extends [UC18](#).
- **Preconditions:** User viewing AI Match Suggestions screen ([UC18](#)).

Basic Flow (Main Success Scenario)

1. User triggers pull-to-refresh gesture or clicks "Refresh Feed" button.
2. System checks rate limit for refresh calls.
3. System fetches next set of unviewed candidate profiles from database.
4. System calculates AI scores ([UC19](#)) and updates UI card deck with animation.

Alternative Flows

- **AF21.1: Refresh Rate Limit Reached**
 - Step 2a: User clicks refresh more than 10 times in 1 minute.
 - Step 2b: System shows toast: "Please wait a moment before refreshing again."
 - **Postconditions:** Candidate feed updated with new recommendations.
 - **UI Prototype Reference:** [!\[Refresh Suggestions Screen\]\(prototypes/uc18_ai_match_suggestions.png\)](#)
-

5. Swipe and Match System (FG-04)

UC22: Like Candidate

- **Use Case ID:** UC22
- **Use Case Name:** Like Candidate
- **Actor(s):** Registered User (Primary), System (Supporting)
- **Description:** Registers a positive "Like" interaction on a candidate profile (swipe right or tap Heart button), checking for a mutual match condition (UC24).
- **Preconditions:** User is viewing a candidate profile card.

Basic Flow (Main Success Scenario)

1. User swipes right on candidate card or clicks "Heart / Like" button.
2. System displays swipe confirmation micro-animation instantly ($\leq 200\text{ms}$).
3. System records "LIKE" interaction record (Liker ID, Target ID, Timestamp) in database.
4. System queries database to check if Target user has previously liked Liker user.
5. If mutual like exists, system triggers UC24 (Create Mutual Match).
6. System removes current card from deck and reveals next candidate.

Alternative Flows

- **AF22.1: Network Error during Swipe**
 - Step 3a: Network connection fails while persisting like action.
 - Step 3b: System queues interaction locally in offline storage and syncs upon reconnection.
 - **Postconditions:** Interaction stored; mutual match created if target previously liked user.
 - **Special Requirements:** Micro-animation feedback $< 200\text{ms}$ (PERF-08 / UX-02).
 - **UI Prototype Reference:** ![Like Candidate Screen]
(prototypes/uc24_mutual_match_modal.png)
-

UC23: Skip Candidate

- **Use Case ID:** UC23
- **Use Case Name:** Skip Candidate
- **Actor(s):** Registered User (Primary)
- **Description:** Registers a negative "Skip" interaction on a candidate profile (swipe left or tap X button), removing the card from feed.
- **Preconditions:** User viewing candidate card.

Basic Flow (Main Success Scenario)

1. User swipes left on candidate card or clicks "X / Skip" button.
2. System displays swipe left micro-animation ($< 200\text{ms}$).
3. System records "SKIP" interaction in database to avoid re-showing profile.

4. System advances deck to display next candidate profile.

Alternative Flows

- **AF23.1: Undo Skip (Premium / Future Extension)**
 - Step 1a: User accidentally skips card.
 - Step 1b: System provides 3-second undo snackbar to revert last skip.
 - **Postconditions:** Candidate skipped and hidden from active discovery deck.
 - **UI Prototype Reference:** ![Skip Candidate Screen]
(prototypes/uc24_mutual_match_modal.png)
-

UC24: Create Mutual Match

- **Use Case ID:** UC24
- **Use Case Name:** Create Mutual Match
- **Actor(s):** System (Primary), Registered User (Secondary)
- **Description:** System background trigger executed when a mutual like is established between two users. Creates match record, triggers notification (UC40), and shows match popup (UC25). Extends UC22.
- **Preconditions:** Both User A and User B have issued a "LIKE" action on each other's profile.

Basic Flow (Main Success Scenario)

1. System detects bidirectional "LIKE" records between User A and User B.
2. System creates new **Match** entry in database (Match ID, User A ID, User B ID, Created At).
3. System unlocks real-time messaging channel between User A and User B.
4. System triggers UC40 (Receive Match Notification) for both users.
5. System invokes UC25 (Show Match Confirmation) modal on active user screen.

Alternative Flows

- **AF24.1: Target User Account Blocked/Deleted**
 - Step 1a: Before match creation completes, target user account becomes blocked or deleted.
 - Step 1b: System cancels match creation silently.
 - **Postconditions:** Match record created in database; conversation channel enabled.
 - **UI Prototype Reference:** ![Create Mutual Match Screen]
(prototypes/uc24_mutual_match_modal.png)
-

UC25: Show Match Confirmation

- **Use Case ID:** UC25
- **Use Case Name:** Show Match Confirmation
- **Actor(s):** Registered User (Primary)

- **Description:** Displays full-screen celebratory "It's a Match!" modal screen with match avatars and quick action buttons ("Send Message" / "Keep Swiping"). Included by UC24.
- **Preconditions:** Mutual match created successfully (UC24).

Basic Flow (Main Success Scenario)

1. System overlays celebratory modal on user screen displaying "It's a Match! 🎉".
2. Modal displays side-by-side profile photos of both matched users.
3. System presents two buttons: "Send Message" and "Keep Swiping".
4. If user taps "Send Message", system routes user to messaging view (UC31/UC32).
5. If user taps "Keep Swiping", system closes modal and resumes candidate deck (UC18).

Alternative Flows

- **AF25.1: Dismiss Modal**
 - Step 3a: User taps backdrop background.
 - Step 3b: Modal closes, returning user to deck.
 - **Postconditions:** User acknowledges match and chooses immediate chat or browsing.
 - **UI Prototype Reference:** ![Show Match Confirmation Screen](prototypes/uc24_mutual_match_modal.png)
-

UC26: View Match History

- **Use Case ID:** UC26
- **Use Case Name:** View Match History
- **Actor(s):** Registered User (Primary)
- **Description:** Displays horizontal scroll view or list of all active mutual matches with avatar thumbnails and match timestamps.
- **Preconditions:** User is logged in.

Basic Flow (Main Success Scenario)

1. User navigates to Matches tab.
2. System queries active matches associated with user ID.
3. System renders list of matched user profiles sorted by match recency.
4. Tapping a match thumbnail opens user profile details or chat thread.

Alternative Flows

- **AF26.1: Zero Matches**
 - Step 2a: User has no active mutual matches yet.
 - Step 2b: System displays motivational empty state: "No matches yet. Keep swiping to find your connection!"
- **Postconditions:** Active matches displayed to user.

- **UI Prototype Reference:** ![View Match History Screen] (prototypes/uc24_mutual_match_modal.png)
-

6. Advanced Search and Filtering (FG-05)

UC27: Search Profiles

- **Use Case ID:** UC27
- **Use Case Name:** Search Profiles
- **Actor(s):** Registered User (Primary)
- **Description:** Allows user to execute custom searches for potential matches by keyword, applying filters and sorting parameters. Includes UC30 and extended by UC28, UC29.
- **Preconditions:** User is logged in.

Basic Flow (Main Success Scenario)

1. User opens the "Search & Explore" tab.
2. System displays search query bar and quick filter chips.
3. User enters keyword (e.g. name or bio keyword) or applies filter parameters (UC28).
4. System executes search query against database profile records.
5. System returns matching candidate profiles using paginated result structure (UC30).
6. System renders profile cards with photo, name, age, city, and interest tags.

Alternative Flows

- **AF27.1: No Search Results Found**
 - Step 4a: Database returns zero matches for query.
 - Step 4b: System displays message: "No profiles matched your search parameters."
 - **Postconditions:** Paginated search results rendered to user.
 - **Special Requirements:** Response time < 2s for paginated results (PERF-13).
 - **UI Prototype Reference:** ![Search Profiles Screen] (prototypes/uc18_ai_match_suggestions.png)
-

UC28: Apply Search Filters

- **Use Case ID:** UC28
- **Use Case Name:** Apply Search Filters
- **Actor(s):** Registered User (Primary)
- **Description:** Configures explicit filter parameters: Target Gender, Age Range (min/max), City/Province, and Specific Interest Tags. Extends UC27.
- **Preconditions:** User in Search & Explore view (UC27).

Basic Flow (Main Success Scenario)

1. User taps "Filter" button.
2. System opens filter modal with parameters: Gender (Male, Female, All), Age slider (18-65+), City dropdown, Interest tag checkboxes.
3. User adjusts filter parameters and taps "Apply Filters".
4. System updates active filter criteria state and re-executes search (UC27).

Alternative Flows

- **AF28.1: Reset Filters**
 - Step 3a: User taps "Reset to Default".
 - Step 3b: System restores default preference filters and updates search results.
 - **Postconditions:** Custom search filters applied to query.
 - **UI Prototype Reference:** ![Apply Search Filters Screen] (prototypes/uc18_ai_match_suggestions.png)
-

UC29: Sort Search Results

- **Use Case ID:** UC29
- **Use Case Name:** Sort Search Results
- **Actor(s):** Registered User (Primary)
- **Description:** Sorts search result cards by "AI Compatibility Score (High to Low)" or "Recently Active / Newest". Extends UC27.
- **Preconditions:** Search result list populated (UC27).

Basic Flow (Main Success Scenario)

1. User selects "Sort By" dropdown menu on search header.
2. User selects sort option (e.g. "Compatibility Score: High to Low").
3. System re-orders result dataset and re-renders profile grid.

Alternative Flows

- **AF29.1: Default Sort**
 - Step 1a: No sort selected by user.
 - Step 1b: System defaults to AI Compatibility Score descending.
 - **Postconditions:** Search results displayed in selected order.
 - **UI Prototype Reference:** ![Sort Search Results Screen] (prototypes/uc18_ai_match_suggestions.png)
-

UC30: View Paginated Results

- **Use Case ID:** UC30
- **Use Case Name:** View Paginated Results

- **Actor(s):** System (Primary), Registered User (Secondary)
- **Description:** Manages paginated profile loading (20 profiles per page / infinite scroll batching) to maintain system performance. Included by [UC27](#).
- **Preconditions:** Search or discovery query executed.

Basic Flow (Main Success Scenario)

1. System receives page request parameter (e.g. [page=1](#), [limit=20](#)).
2. System retrieves slice of 20 profile records from database.
3. System appends profiles to search view grid.
4. When user scrolls to bottom of list, system automatically fetches page 2.

Alternative Flows

- **AF30.1: End of Result Set**
 - Step 4a: Subsequent page request returns 0 records.
 - Step 4b: System disables infinite scroll listener and displays: "You have reached the end of the results."
- **Postconditions:** Smooth paginated rendering of candidate profiles achieved.
- **Special Requirements:** Max 20 items per page batch ([PERF-13](#)).
- **UI Prototype Reference:** [!\[View Paginated Results Screen\]\(prototypes/uc18_ai_match_suggestions.png\)](#)

7. Real-time Messaging (FG-06)

Assignee	Reviewer	Editor
Nguyễn Minh Hoàng	Lê Văn Hoàng Tấn	Trần Đại Nghĩa

UC31: View Conversations

- **Use Case ID:** UC31
- **Use Case Name:** View Conversations
- **Actor(s):** Registered User (Primary)
- **Description:** Renders conversation inbox showing all active chat channels with matched users, snippet of latest message, timestamp, unread badge, online status, and typing indicator. Includes [UC34](#), [UC35](#), [UC36](#).
- **Preconditions:** User is logged in with active matches.

Basic Flow (Main Success Scenario)

1. User clicks "Messages / Chat" main tab.
2. System fetches all active conversation channels associated with user.

3. System invokes **UC34** (View Message History snippet), **UC35** (View Online Status), and **UC36** (View Typing Indicator).
4. System renders conversation list sorted by most recent message timestamp.
5. User selects a conversation thread to open full chat interface.

Alternative Flows

- **AF31.1: No Active Conversations**
 - Step 2a: User has matches but no initiated chats.
 - Step 2b: System shows top horizontal row of match avatars prompting user to "Say Hello!".
 - **Postconditions:** User inbox rendered with real-time status indicators.
 - **UI Prototype Reference:** [!\[View Conversations Screen\]\(prototypes/uc31_realtime_chat.png\)](#)
-

UC32: Send Message

- **Use Case ID:** UC32
- **Use Case Name:** Send Message
- **Actor(s):** Registered User (Primary)
- **Description:** Transmits text message to matched user over WebSocket real-time channel with fallback retries.
- **Preconditions:** Mutual match established between sender and recipient; active chat thread open.

Basic Flow (Main Success Scenario)

1. User types message text into chat input box.
2. User taps "Send" button or presses Enter.
3. Client validates message content non-empty and length ≤ 1000 characters.
4. Client transmits message payload via WebSocket connection ([/ws/chat](#)).
5. Server persists message in database, updates conversation **last_message_at**, and echoes message receipt.
6. Server forwards message payload instantly to recipient socket (**UC33**).
7. Client renders sent message bubble with "Sent" status indicator.

Alternative Flows

- **AF32.1: WebSocket Disconnection / Retry (PLAT-06)**
 - Step 4a: WebSocket connection is disconnected when user sends message.
 - Step 4b: System attempts up to 3 automatic retries over 5 seconds.
 - Step 4c: If retries fail, message bubble shows "Failed to send. Tap to retry" error indicator.
- **AF32.2: Recipient Unmatched or Blocked**
 - Step 5a: Recipient has blocked user (**UC42**) or unmatched.
 - Step 5b: Server rejects message delivery with error code: "Cannot send message to this user."

- **Postconditions:** Message persisted and delivered to recipient; conversation timestamp updated.
 - **Special Requirements:** End-to-end latency < 300ms (**PERF-05**); WebSocket auto-reconnect (**PLAT-06**).
 - **UI Prototype Reference:** ![Send Message Screen](prototypes/uc31_realtime_chat.png)
-

UC33: Receive Message

- **Use Case ID:** UC33
- **Use Case Name:** Receive Message
- **Actor(s):** Registered User (Primary)
- **Description:** Receives incoming WebSocket text message event from matched sender, updating active thread UI and issuing notification if chat inactive.
- **Preconditions:** Mutual match active; WebSocket connection connected.

Basic Flow (Main Success Scenario)

1. Recipient client receives incoming **CHAT_MESSAGE** event over WebSocket.
2. If recipient is currently viewing sender's chat thread, client appends message bubble to chat window with sound effect.
3. If recipient is on a different screen, system triggers **UC41** (Receive Message Notification) to display unread toast and badge counter increment.

Alternative Flows

- **AF33.1: Client Offline upon Transmission**
 - Step 1a: Recipient socket is disconnected.
 - Step 1b: Server stores message in DB and queues push notification (**UC41**). Message renders when client reconnects.
 - **Postconditions:** Incoming message displayed to recipient.
 - **Special Requirements:** Delivery latency < 300ms (**PERF-05**).
 - **UI Prototype Reference:** ![Receive Message Screen](prototypes/uc31_realtime_chat.png)
-

UC34: View Message History

- **Use Case ID:** UC34
- **Use Case Name:** View Message History
- **Actor(s):** Registered User (Primary)
- **Description:** Fetches and displays chronological chat log history for a specific conversation channel with pagination on upward scroll. Included by **UC31**.
- **Preconditions:** Active conversation selected.

Basic Flow (Main Success Scenario)

1. User selects a chat thread.

2. System loads latest 50 messages for conversation.
3. System renders message bubbles grouped by date headers.
4. User scrolls up to view older messages.
5. System fetches next 50 historical messages page and prepends to scroll view.

Alternative Flows

- **AF34.1: End of Message History**
 - Step 5a: No further historical messages exist.
 - Step 5b: System displays top marker: "Beginning of conversation".
 - **Postconditions:** Message history rendered to user.
 - **UI Prototype Reference:** ![View Message History Screen](prototypes/uc31_realtime_chat.png)
-

UC35: View Online Status

- **Use Case ID:** UC35
- **Use Case Name:** View Online Status
- **Actor(s):** Registered User (Primary)
- **Description:** Displays real-time presence status (Green dot for Online, "Active X mins ago" for Offline) next to matched user avatar. Included by **UC31**.
- **Preconditions:** User viewing conversation list or chat window.

Basic Flow (Main Success Scenario)

1. System subscribes to presence state of matched users via WebSocket presence channel.
2. System renders green status indicator dot for online users.
3. When matched user disconnects, system updates status to "Active 5m ago".

Alternative Flows

- **AF35.1: Privacy Masking**
 - Step 2a: User or match has hidden online status in privacy settings.
 - Step 2b: System suppresses status indicator display.
 - **Postconditions:** Online status accurately reflected.
 - **UI Prototype Reference:** ![View Online Status Screen](prototypes/uc31_realtime_chat.png)
-

UC36: View Typing Indicator

- **Use Case ID:** UC36
- **Use Case Name:** View Typing Indicator
- **Actor(s):** Registered User (Primary)

- **Description:** Displays animated "User is typing..." indicator when matched user is actively typing a message in the thread. Included by UC31.
- **Preconditions:** Chat thread open between matched users.

Basic Flow (Main Success Scenario)

1. Matched user inputs keystrokes in chat field, broadcasting TYPING_START socket event.
2. Recipient client receives event and renders three-dot typing animation below chat log.
3. When matched user stops typing for 3 seconds or sends message, TYPING_STOP event clears animation.

Alternative Flows

- **AF36.1: Socket Timeout**
 - Step 3a: No TYPING_STOP received after 5 seconds.
 - Step 3b: Client automatically hides typing indicator.
- **Postconditions:** Typing status dynamically displayed.
- **UI Prototype Reference:** ![View Typing Indicator Screen] (prototypes/uc31_realtime_chat.png)

8. Notification Center (FG-07)

Assignee	Reviewer	Editor
Nguyễn Minh Hoàng	Lê Văn Hoàng Tấn	Trần Đại Nghĩa

UC37: View Notifications

- **Use Case ID:** UC37
- **Use Case Name:** View Notifications
- **Actor(s):** Registered User (Primary)
- **Description:** Renders central notifications drop-down list showing alerts for new matches, new messages, and system announcements. Extended by UC38, UC39.
- **Preconditions:** User is logged in.

Basic Flow (Main Success Scenario)

1. User clicks Bell icon in navigation header.
2. System fetches user notification records from database.
3. System renders notification item list showing type icon, title, description, timestamp, and read status indicator.
4. Clicking a notification redirects user to corresponding resource (e.g. Chat thread or Match confirmation).

Alternative Flows

- **AF37.1: Empty Notification Center**
 - Step 2a: User has no notifications.
 - Step 2b: System renders: "You're all caught up! No new notifications."
 - **Postconditions:** User views list of alerts; unread badge state managed.
 - **Special Requirements:** Real-time badge update without page refresh (UX-04).
 - **UI Prototype Reference:** ![View Notifications Screen](prototypes/uc31_realtime_chat.png)
-

UC38: Mark Notification as Read

- **Use Case ID:** UC38
- **Use Case Name:** Mark Notification as Read
- **Actor(s):** Registered User (Primary)
- **Description:** Updates status of single notification item from UNREAD to READ upon click or manual action. Extends UC37.
- **Preconditions:** User viewing notifications list (UC37).

Basic Flow (Main Success Scenario)

1. User taps an unread notification item.
2. System issues request to mark notification ID as READ.
3. System updates item styling (removes highlight) and decrements unread counter badge.

Alternative Flows

- **AF38.1: Already Read**
 - Step 1a: User taps an item already marked READ.
 - Step 1b: System navigates to resource without sending state update request.
 - **Postconditions:** Target notification status updated to READ in database.
 - **UI Prototype Reference:** ![Mark Notification as Read Screen](prototypes/uc31_realtime_chat.png)
-

UC39: Mark All Notifications as Read

- **Use Case ID:** UC39
- **Use Case Name:** Mark All Notifications as Read
- **Actor(s):** Registered User (Primary)
- **Description:** Bulk updates all unread notifications associated with user to READ status. Extends UC37.
- **Preconditions:** User viewing notifications list (UC37).

Basic Flow (Main Success Scenario)

1. User clicks "Mark All as Read" button at top of notification drawer.
2. System sends bulk update request for user ID.
3. System updates database records, updates UI items to read state, and resets header unread badge counter to 0.

Alternative Flows

- **AF39.1: No Unread Notifications**
 - Step 1a: Unread count is already 0.
 - Step 1b: Button is disabled.
 - **Postconditions:** All notifications marked **READ**; unread badge cleared.
 - **UI Prototype Reference:** [!\[Mark All as Read Screen\]\(prototypes/uc31_realtime_chat.png\)](#)
-

UC40: Receive Match Notification

- **Use Case ID:** UC40
- **Use Case Name:** Receive Match Notification
- **Actor(s):** Registered User (Primary), System (Supporting)
- **Description:** System-generated real-time push/in-app alert dispatched when a mutual match is formed ([UC24](#)).
- **Preconditions:** Mutual match created ([UC24](#)).

Basic Flow (Main Success Scenario)

1. System creates new notification record of type **NEW_MATCH**.
2. System pushes real-time WebSocket event **NOTIFICATION_RECEIVED** to online user.
3. Client renders toast notification: "You matched with [Name]! 🏆".
4. System increments unread notification badge counter by 1.

Alternative Flows

- **AF40.1: User Offline**
 - Step 2a: User is offline.
 - Step 2b: Notification stored in database and shown on next log in.
 - **Postconditions:** Match alert delivered to user.
 - **UI Prototype Reference:** [!\[Receive Match Notification Screen\]\(prototypes/uc31_realtime_chat.png\)](#)
-

UC41: Receive Message Notification

- **Use Case ID:** UC41
- **Use Case Name:** Receive Message Notification
- **Actor(s):** Registered User (Primary), System (Supporting)

- **Description:** Dispatches notification alert when user receives a new message while outside that specific chat room.
- **Preconditions:** New chat message sent ([UC32/UC33](#)).

Basic Flow (Main Success Scenario)

1. System detects incoming message for user who is not currently in sender's active chat window.
2. System creates notification record of type [NEW_MESSAGE](#).
3. System displays top banner toast: "[Name]: [Message snippet]".
4. System increments header chat badge counter.

Alternative Flows

- **AF41.1: Muted Conversation**
 - Step 1a: User has muted notifications for this specific match.
 - Step 1b: System increments unread badge silently without banner toast.
- **Postconditions:** Message notification delivered.
- **UI Prototype Reference:** [!\[Receive Message Notification Screen\]\(prototypes/uc31_realtime_chat.png\)](#)

9. Privacy and Safety Controls (FG-08)

UC42: Block User

- **Use Case ID:** UC42
- **Use Case Name:** Block User
- **Actor(s):** Registered User (Primary)
- **Description:** Allows user to block another profile, immediately terminating mutual match status, hiding profiles from each other, and preventing all communication.
- **Preconditions:** User viewing target profile or active chat thread.

Basic Flow (Main Success Scenario)

1. User clicks "..." action menu on target user profile or chat window.
2. User selects "Block User".
3. System displays confirmation modal: "Block [Name]? They will not be able to see your profile or message you."
4. User confirms "Block".
5. System creates [Block](#) record in database (Blocker ID, Blocked ID, Timestamp).
6. System revokes mutual match status, deletes chat channel access, and removes target profile from user's views.
7. System displays success notice: "User blocked".

Alternative Flows

- **AF42.1: Cancel Block**
 - Step 4a: User selects "Cancel".
 - Step 4b: System closes modal without modifying block list.
 - **Postconditions:** User blocked; communication channels destroyed; profiles mutually invisible ([SEC-12](#)).
 - **UI Prototype Reference:** [!\[Block User Screen\]\(prototypes/uc31_realtime_chat.png\)](#)
-

UC43: Report User

- **Use Case ID:** UC43
- **Use Case Name:** Report User
- **Actor(s):** Registered User (Primary)
- **Description:** Allows user to report another user for violations (harassment, fake profile, inappropriate photos, spam) for admin review ([UC48/UC50](#)).
- **Preconditions:** User logged in.

Basic Flow (Main Success Scenario)

1. User opens action menu on candidate profile or chat and selects "Report User".
2. System presents modal requesting report reason category (Harassment, Fake Profile, Inappropriate Content, Spam) and optional text description.
3. User selects category, adds detail notes, and taps "Submit Report".
4. System creates [Report](#) ticket in database with status [PENDING](#) attached to target user ID.
5. System displays prompt: "Report submitted. Would you also like to block this user?"
6. If confirmed, system executes [UC42](#) (Block User).

Alternative Flows

- **AF43.1: Missing Reason Category**
 - Step 3a: User attempts submission without selecting a category.
 - Step 3b: System highlights required category selection.
 - **Postconditions:** Moderation ticket generated for Admin review in Admin Dashboard ([UC48](#)).
 - **UI Prototype Reference:** [!\[Report User Screen\]\(prototypes/uc31_realtime_chat.png\)](#)
-

UC44: Deactivate Account

- **Use Case ID:** UC44
- **Use Case Name:** Deactivate Account
- **Actor(s):** Registered User (Primary)
- **Description:** Allows user to temporarily deactivate account, hiding profile from candidate feeds while preserving account data for future reactivation.
- **Preconditions:** User is logged in.

Basic Flow (Main Success Scenario)

1. User opens Account Privacy Settings and selects "Deactivate Account".
2. System displays warning modal explaining that profile will be hidden from matching discovery.
3. User enters current password for verification.
4. System validates password, updates account status to **DEACTIVATED**, and terminates session (**UC03**).
5. System redirects to login page with message: "Your account is deactivated. Log in anytime to reactivate."

Alternative Flows

- **AF44.1: Invalid Password Verification**
 - Step 4a: Password check fails.
 - Step 4b: System displays error and denies deactivation.
 - **Postconditions:** Account state set to **DEACTIVATED**; profile hidden from feeds.
 - **UI Prototype Reference:** [!\[Deactivate Account Screen\]\(prototypes/uc31_realtime_chat.png\)](#)
-

UC45: Permanently Delete Account

- **Use Case ID:** UC45
- **Use Case Name:** Permanently Delete Account
- **Actor(s):** Registered User (Primary)
- **Description:** Permanently purges user account, profile, uploaded photos, match records, and chat history within 24 hours in compliance with privacy regulations (**PRIV-01**).
- **Preconditions:** User is logged in.

Basic Flow (Main Success Scenario)

1. User opens Security Settings and selects "Delete Account Permanently".
2. System displays strong confirmation modal warning: "This action is irreversible. All photos, matches, and messages will be permanently deleted within 24 hours."
3. User types "DELETE" into text box and enters current password.
4. System validates password and confirmation text.
5. System schedules permanent data purge job ($TTL \leq 24h$) and marks account **PENDING_DELETION**.
6. System terminates user session (**UC03**) and displays completion notification.

Alternative Flows

- **AF45.1: Mismatched Confirmation String**
 - Step 4a: Confirmation string does not match "DELETE".
 - Step 4b: System disables deletion button.
- **Postconditions:** User data queued for total removal within 24 hours (**PRIV-01**).
- **Special Requirements:** Full data purge within 24 hours (**PRIV-01**).

- **UI Prototype Reference:** ![Permanently Delete Account Screen] (prototypes/uc31_realtime_chat.png)
-

10. Administration (FG-09)

UC46: View Admin Dashboard

- **Use Case ID:** UC46
- **Use Case Name:** View Admin Dashboard
- **Actor(s):** Administrator (Primary)
- **Description:** Provides central administrative command view displaying platform telemetry metrics, pending reports queue, and management modules. Includes UC47.
- **Preconditions:** Admin user logged in with ROLE_ADMIN (UC02/UC06).

Basic Flow (Main Success Scenario)

1. Admin user logs into administrative portal.
2. System verifies ROLE_ADMIN authorization (UC06).
3. System invokes UC47 (View Platform Statistics).
4. System renders dashboard layout displaying metric cards (Total Users, Active Users Today, Total Matches, Pending Report Tickets), recent user registration chart, and management navigation menu.

Alternative Flows

- **AF46.1: Unauthorized Access Attempt (SEC-11)**
 - Step 2a: Non-admin token attempts to access /admin/dashboard.
 - Step 2b: System blocks access with HTTP 403 Forbidden and logs security incident (SEC-13).
 - **Postconditions:** Admin metrics and management interface displayed.
 - **UI Prototype Reference:** ![View Admin Dashboard Screen] (prototypes/uc46_admin_dashboard.png)
-

UC47: View Platform Statistics

- **Use Case ID:** UC47
- **Use Case Name:** View Platform Statistics
- **Actor(s):** Administrator (Primary), System (Supporting)
- **Description:** Aggregates and renders real-time platform performance statistics (Total Users, Daily Active Users, Match Rate, Reported Accounts). Included by UC46.
- **Preconditions:** Admin dashboard requested (UC46).

Basic Flow (Main Success Scenario)

1. System queries database analytics views for total user count, DAU, match counts, and pending report tickets.

2. System computes platform health metrics.
3. System renders summary metric cards and visual bar charts on dashboard.

Alternative Flows

- **AF47.1: Analytics Service Delay**
 - Step 1a: Aggregation query takes > 2s.
 - Step 1b: System displays cached statistics snapshot with timestamp indicator.
 - **Postconditions:** Platform telemetry visualized for Admin.
 - **UI Prototype Reference:** ![View Platform Statistics Screen](prototypes/uc46_admin_dashboard.png)
-

UC48: Manage Users

- **Use Case ID:** UC48
- **Use Case Name:** Manage Users
- **Actor(s):** Administrator (Primary)
- **Description:** Provides administrative user management portal to search, filter, inspect details, block, unblock, or delete user accounts. Includes UC49, UC50.
- **Preconditions:** Admin logged in.

Basic Flow (Main Success Scenario)

1. Admin clicks "User Management" tab on sidebar menu.
2. System executes UC49 (Search Users) and renders paginated list of user accounts with columns: User ID, Name, Email, Status (ACTIVE/BLOCKED/DEACTIVATED), Date Joined, and Actions.
3. Admin clicks a user row to invoke UC50 (View User Details).

Alternative Flows

- **AF48.1: Filter by Status**
 - Step 2a: Admin selects filter dropdown (e.g. "Status: Pending Reports").
 - Step 2b: System filters list to show only accounts with pending report tickets.
 - **Postconditions:** Account list rendered for administration.
 - **UI Prototype Reference:** ![Manage Users Screen](prototypes/uc46_admin_dashboard.png)
-

UC49: Search Users

- **Use Case ID:** UC49
- **Use Case Name:** Search Users
- **Actor(s):** Administrator (Primary)
- **Description:** Allows Admin to search user accounts by ID, display name, or email address. Included by UC48.

- **Preconditions:** Admin in User Management view (UC48).

Basic Flow (Main Success Scenario)

1. Admin types search term (e.g. email or username) into Admin Search bar.
2. System executes database search query against user records.
3. System displays matching accounts list.

Alternative Flows

- **AF49.1: No Accounts Found**
 - Step 2a: No matching account records.
 - Step 2b: System displays: "No users found matching search term."
 - **Postconditions:** Target user accounts filtered for Admin.
 - **UI Prototype Reference:** ![Search Users Screen](prototypes/uc46_admin_dashboard.png)
-

UC50: View User Details

- **Use Case ID:** UC50
- **Use Case Name:** View User Details
- **Actor(s):** Administrator (Primary)
- **Description:** Renders complete profile audit view for a specific user, including account history, uploaded photos, report tickets, and action triggers (UC51, UC52, UC53). Included by UC48.
- **Preconditions:** Admin in User Management view (UC48).

Basic Flow (Main Success Scenario)

1. Admin selects a user account from user table.
2. System renders comprehensive details panel: Account Metadata, Profile Photos, Activity Logs, and Associated Report Tickets.
3. System displays administrative action buttons: "Block Account", "Unblock Account", and "Delete Account".

Alternative Flows

- **AF50.1: Execute Account Block (UC51)**
 - Step 3a: Admin selects "Block Account" to trigger UC51.
- **AF50.2: Execute Account Unblock (UC52)**
 - Step 3a: Admin selects "Unblock Account" to trigger UC52.
- **AF50.3: Execute Account Delete (UC53)**
 - Step 3a: Admin selects "Delete Account" to trigger UC53.
- **Postconditions:** Complete audit details displayed to Admin.

- **UI Prototype Reference:** ![View User Details Screen] (prototypes/uc46_admin_dashboard.png)
-

UC51: Block User Account

- **Use Case ID:** UC51
- **Use Case Name:** Block User Account
- **Actor(s):** Administrator (Primary)
- **Description:** Allows Admin to administratively block a user account due to community violations, revoking active sessions and locking login. Extends UC50.
- **Preconditions:** Admin viewing user details (UC50).

Basic Flow (Main Success Scenario)

1. Admin clicks "Block Account" button on user details view.
2. System prompts Admin for reason for administrative ban (e.g. Harassment, Fake Account) and audit notes.
3. Admin inputs reason and clicks "Confirm Block".
4. System updates account status to **BLOCKED** in database.
5. System revokes all active JWT tokens for target user and logs audit record (SEC-13).
6. System displays confirmation: "User account blocked successfully".

Alternative Flows

- **AF51.1: Missing Ban Reason**
 - Step 3a: Admin submits without specifying a reason note.
 - Step 3b: System prompts: "Audit reason required for account ban."
 - **Postconditions:** User account blocked; active session revoked; audit log recorded (SEC-13).
 - **Special Requirements:** Audit log record required (SEC-13).
 - **UI Prototype Reference:** ![Block User Account Screen] (prototypes/uc46_admin_dashboard.png)
-

UC52: Unblock User Account

- **Use Case ID:** UC52
- **Use Case Name:** Unblock User Account
- **Actor(s):** Administrator (Primary)
- **Description:** Allows Admin to restore a previously blocked user account to **ACTIVE** status. Extends UC50.
- **Preconditions:** Admin viewing details of a **BLOCKED** account (UC50).

Basic Flow (Main Success Scenario)

1. Admin clicks "Unblock Account" button on user details view.

2. System presents confirmation modal: "Unblock user [Name] and restore access?"
3. Admin enters unblock justification note and clicks "Confirm Unblock".
4. System updates account status to **ACTIVE** and logs audit entry (**SEC-13**).
5. System displays notice: "User account unblocked successfully".

Alternative Flows

- **AF52.1: Account Already Active**
 - Step 1a: Target account is not blocked.
 - Step 1b: Unblock button is disabled.
 - **Postconditions:** Account state restored to **ACTIVE**; audit entry logged (**SEC-13**).
 - **UI Prototype Reference:** ![Unblock User Account Screen]
([prototypes/uc46_admin_dashboard.png](#))
-

UC53: Delete User Account

- **Use Case ID:** UC53
- **Use Case Name:** Delete User Account
- **Actor(s):** Administrator (Primary)
- **Description:** Allows Admin to administratively delete a user account and purge user data from system. Extends **UC50**.
- **Preconditions:** Admin viewing user details (**UC50**).

Basic Flow (Main Success Scenario)

1. Admin clicks "Delete User Account" button on user details view.
2. System presents warning dialog requiring Admin password confirmation.
3. Admin enters password and clicks "Confirm Permanent Deletion".
4. System verifies Admin password, deletes user record and associated assets, and logs audit record (**SEC-13**).
5. System displays notification: "User account permanently deleted".

Alternative Flows

- **AF53.1: Incorrect Admin Password**
 - Step 4a: Password check fails.
 - Step 4b: System denies deletion action.
 - **Postconditions:** Target user data purged from system database; audit logged (**SEC-13**).
 - **UI Prototype Reference:** ![Delete User Account Screen]
([prototypes/uc46_admin_dashboard.png](#))
-

UC54: Manage Interest Tag Catalogue

- **Use Case ID:** UC54
- **Use Case Name:** Manage Interest Tag Catalogue
- **Actor(s):** Administrator (Primary)
- **Description:** Provides management view for global interest tag catalogue (e.g. Travel, Music, Gaming, Coffee), allowing creation and removal of tags. Includes UC55, UC56.
- **Preconditions:** Admin logged in.

Basic Flow (Main Success Scenario)

1. Admin selects "Interest Tags" menu item.
2. System loads and renders current tag catalogue grouped by categories (e.g. Lifestyle, Sports, Entertainment).
3. Admin executes UC55 (Add Interest Tag) or UC56 (Remove Interest Tag).
4. System updates global catalogue view.

Alternative Flows

- **AF54.1: Empty Category**
 - Step 2a: Category has no tags.
 - Step 2b: System displays empty category container with "Add Tag" button.
 - **Postconditions:** Interest tag catalogue displayed for management.
 - **UI Prototype Reference:** ![Manage Interest Tag Catalogue Screen] (prototypes/uc46_admin_dashboard.png)
-

UC55: Add Interest Tag

- **Use Case ID:** UC55
- **Use Case Name:** Add Interest Tag
- **Actor(s):** Administrator (Primary)
- **Description:** Allows Admin to add a new interest tag name and category to global catalogue. Included by UC54.
- **Preconditions:** Admin viewing tag catalogue (UC54).

Basic Flow (Main Success Scenario)

1. Admin clicks "Add New Tag" button.
2. System presents modal requesting Tag Name (Vietnamese/English) and Category selection.
3. Admin enters tag details (e.g. "Board Games", category "Entertainment") and submits.
4. System verifies tag name uniqueness.
5. System inserts tag into catalogue database table.
6. System refreshes tag list view.

Alternative Flows

- **AF55.1: Duplicate Tag Name**

- Step 4a: Tag name already exists in catalogue.
 - Step 4b: System displays error: "Interest tag already exists in catalogue."
 - **Postconditions:** New interest tag added to global catalogue.
 - **UI Prototype Reference:** ![Add Interest Tag Screen](prototypes/uc46_admin_dashboard.png)
-

UC56: Remove Interest Tag

- **Use Case ID:** UC56
- **Use Case Name:** Remove Interest Tag
- **Actor(s):** Administrator (Primary)
- **Description:** Allows Admin to archive or remove an interest tag from global catalogue. Included by UC54.
- **Preconditions:** Admin viewing tag catalogue (UC54).

Basic Flow (Main Success Scenario)

1. Admin clicks "Delete" icon next to target interest tag.
2. System displays confirmation modal asking whether to remove tag from catalogue.
3. Admin confirms deletion.
4. System soft-deletes tag record from active catalogue while retaining historical profile links.
5. System updates catalogue UI.

Alternative Flows

- **AF56.1: Cancel Removal**
 - Step 3a: Admin cancels prompt.
 - Step 3b: Tag remains active in catalogue.
- **Postconditions:** Interest tag removed from active global selection list.
- **UI Prototype Reference:** ![Remove Interest Tag Screen](prototypes/uc46_admin_dashboard.png)